



SAMSUNG

NOVA EXECUTIVE
EDUCATION
NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Samsung Innovation Campus

Chapter 9. Deep Learning Module

Word Embeddings:

A (tiny) recap

Word Embeddings: How?

Basically, **word embeddings** represent each word as a **dense** vector that **captures semantic information**.

But how?

Words with similar meanings occur in similar contexts

COUNT

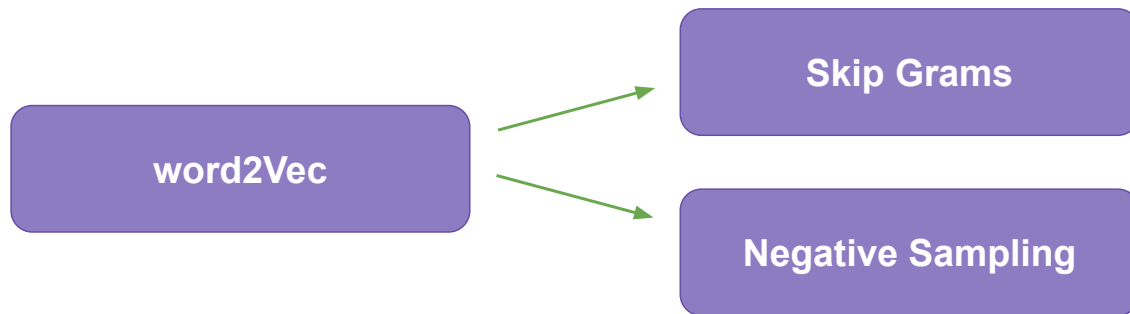


PREDICT

Word Embeddings: How?

Word Embeddings: Dense vectors with continuous values that are trained by exposing a model to a large amount of text data and adjusting the vector representations based on the context in which words appear.

- Encode semantic meaning
- Capture relationships between words.



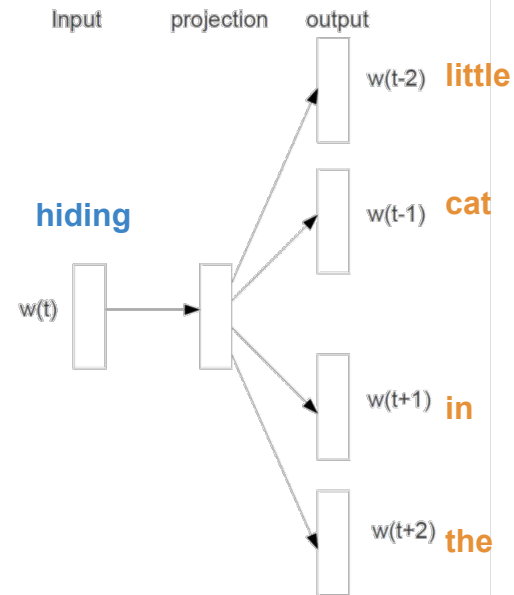
Word Embeddings: Skip-grams

Goal: Predict the **context words** given a **target** word.

The model will maximize the log-likelihood:

$$\frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t)$$

A little cat **hiding** in the tree close by.



Word Embeddings: Negative Sampling

Positive Samples: maximize the likelihood of the pairs

- Words that occur in the neighborhood

“card” → “bank”; “credit”; “debit”; “game”; “student”

Negative Samples: minimize the likelihood of the pairs

- Randomly sampled from the corpus

“card” → “sky”; “mine”; “headphones”; “hat”

Facilitates training

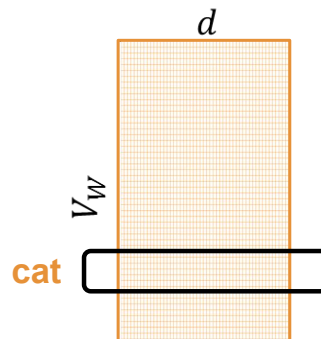
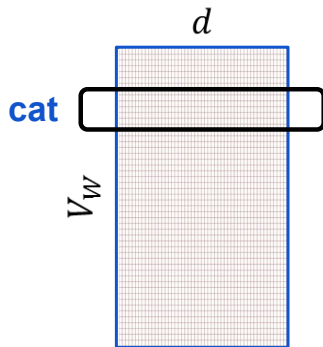
Discriminative

Rare

Word Embeddings: How?

Skip-grams + Negative Sampling finds a dense **vector word** for each word in the vocabulary (V_w).

- Each vector has d latent dimensions (usually 300)
- It learns **TWO** matrices simultaneously: a **word** matrix and a **context** matrix



Word Embeddings: For what?

These models are incredibly good at capturing **notions of similarity**! We can use vector operations in the embedding space to check for similarity analogies!

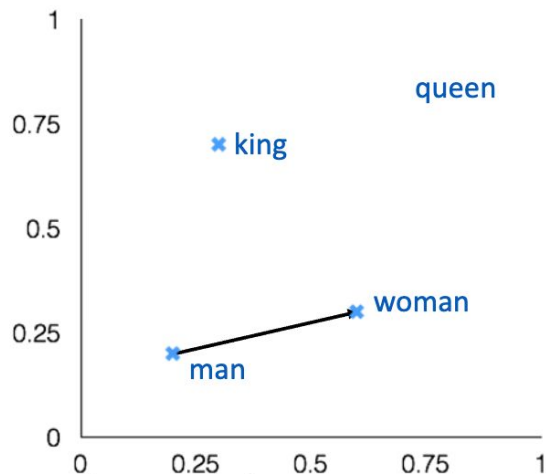
$$\bullet x_{apple} - x_{apples} \approx x_{car} - x_{cars} \approx x_{family} - x_{families}$$

+	king	[0.30 0.70]
---	------	-------------

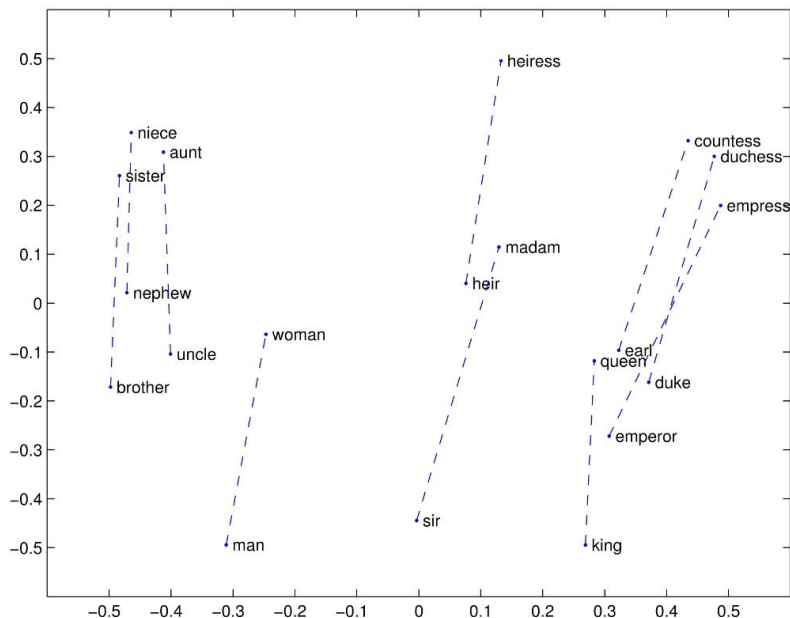
-	man	[0.20 0.20]
---	-----	-------------

+	woman	[0.60 0.30]
---	-------	-------------

	queen	[0.70 0.80]
--	-------	-------------



Word Embeddings: For what?



Word Embeddings: Limitations

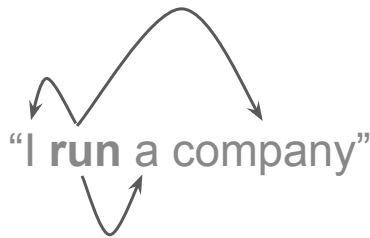
1. **Static Embeddings:** Word sense disambiguation; polysemy
2. **Word tokens:** Does not incorporate subword information - raises issues with **out-of-vocabulary** words
3. **Context:** It only learns from word co-occurrences! It does not capture context-dependent meaning

“I **run** a company”

“I **run** every morning”

Contextual Embeddings: The Novelty

Static word embeddings do not capture the context in which the word appears. However, the **same** word in different contexts may have different **meaning/purposes**.



“I run a company”



“I run every morning”

Each word vector needs to attend to its context

Attention Mechanism

Transformer: Part I

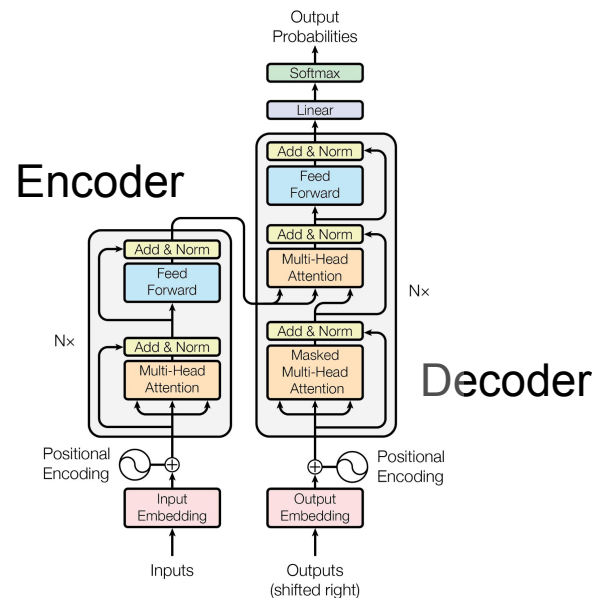


Transformer

The transformer explores the **self-attention** mechanism.

- Computes the importance of each word **in relation** to all others in the input sequence.
- Computed over the entire sequence, in both directions.

Foundation for modern NLP models.





Encoder: An Overview

Input Processing

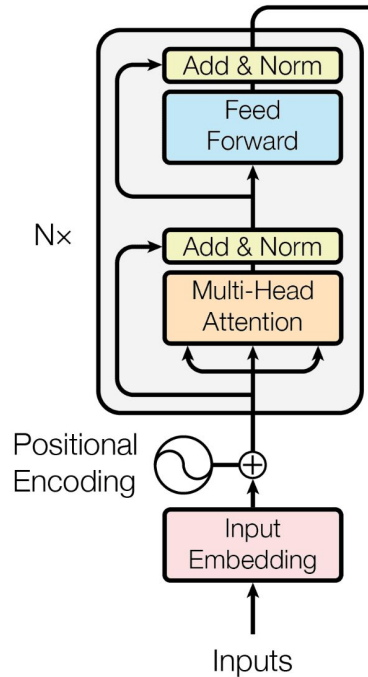
- Tokenization
- Word embeddings
- Positional embeddings

Encoder Layers

- Self-Attention
- Multi-head Self-Attention
- Normalization + Residuals
- Feed Forward

Outputs

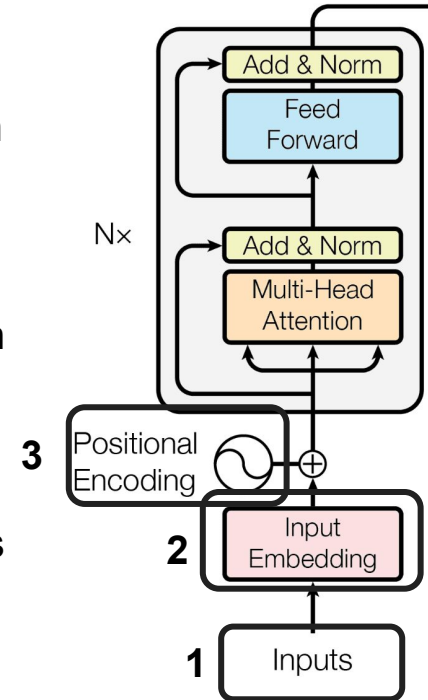
- Contextual embeddings
- Special tokens





Input Processing: Pipeline

1. **Tokenization:** The text is tokenized using a data-drive tokenization approach (like BPE!)
2. **Word Embeddings:** Obtain a dense vector representation for each token
3. **Positional Encodings:** Adds positional information to the embeddings



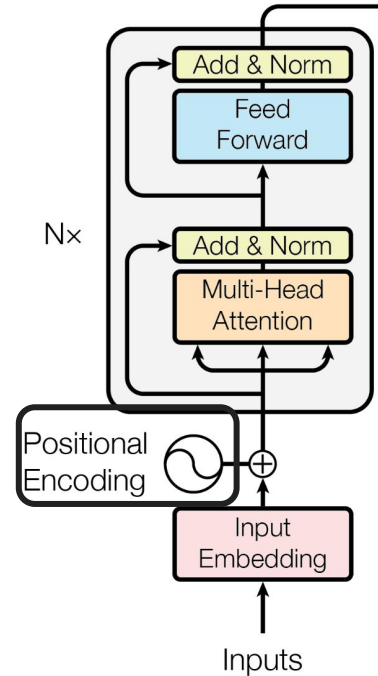


Input Processing: Positional Encodings

The **word embeddings** do not contain information regarding **position!**

We need **positional encodings** that contain sequence information. These are computed from the position in the sequence and the dimension of the vector

We **sum** the positional encodings to the word embedding vectors in order to move them **according to their position** in the sequence





Input Processing: Positional Encodings

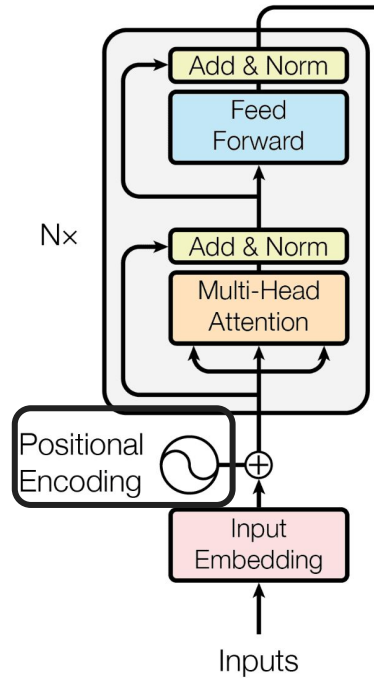
The **word embeddings** do not contain information regarding **position**!

We need **positional encodings** that contain sequence information. These are computed from the position in the sequence and the dimension of the vector

$$PE(i, 2k) = \sin \left(\frac{i}{10000^{\frac{2k}{d}}} \right)$$

$$PE(i, 2k + 1) = \cos \left(\frac{i}{10000^{\frac{2k}{d}}} \right)$$

- i is the position in the sequence.
- $2k$ refers to even dimensions.
- $2k + 1$ refers to odd dimensions.
- d is the total dimension of the position embedding.

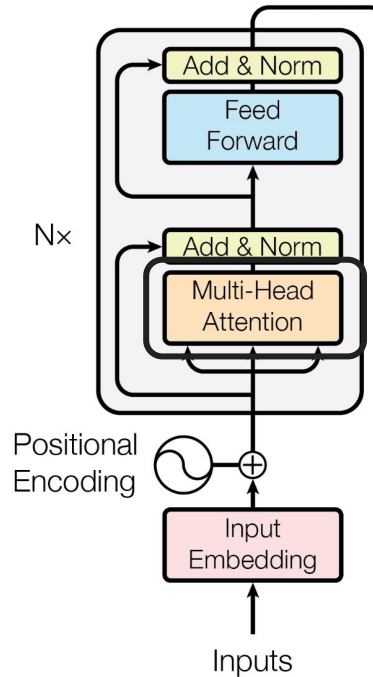




Encoder Layers: Self-Attention

- Capture **long-range dependencies** and **relationships** within input sequences
- Allows **updating** the input embeddings by integrating relevant information from other tokens in the sequence.

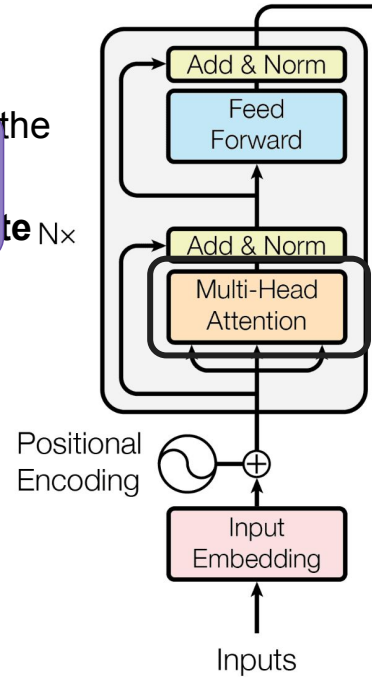
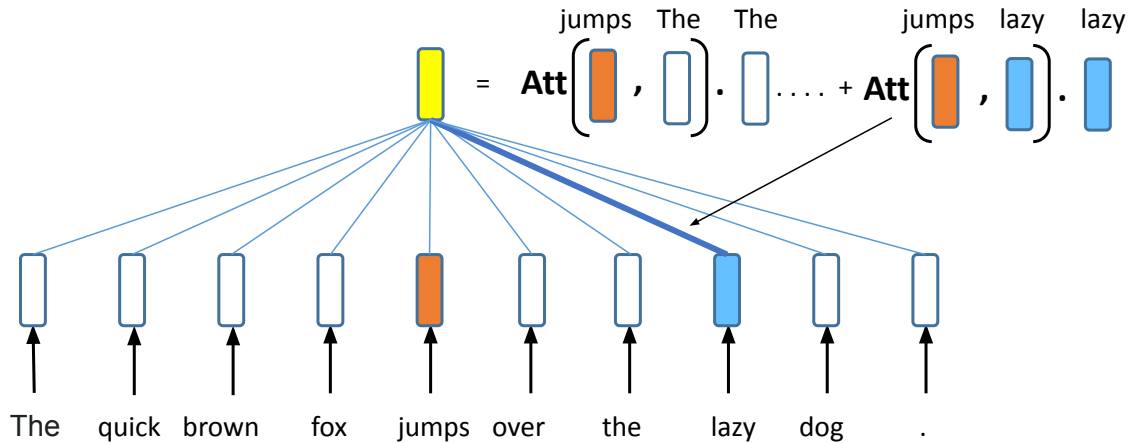
Self-Attention vs Attention: Attends to the input sequence itself instead of attending to an external context





Encoder Layers: Self-Attention

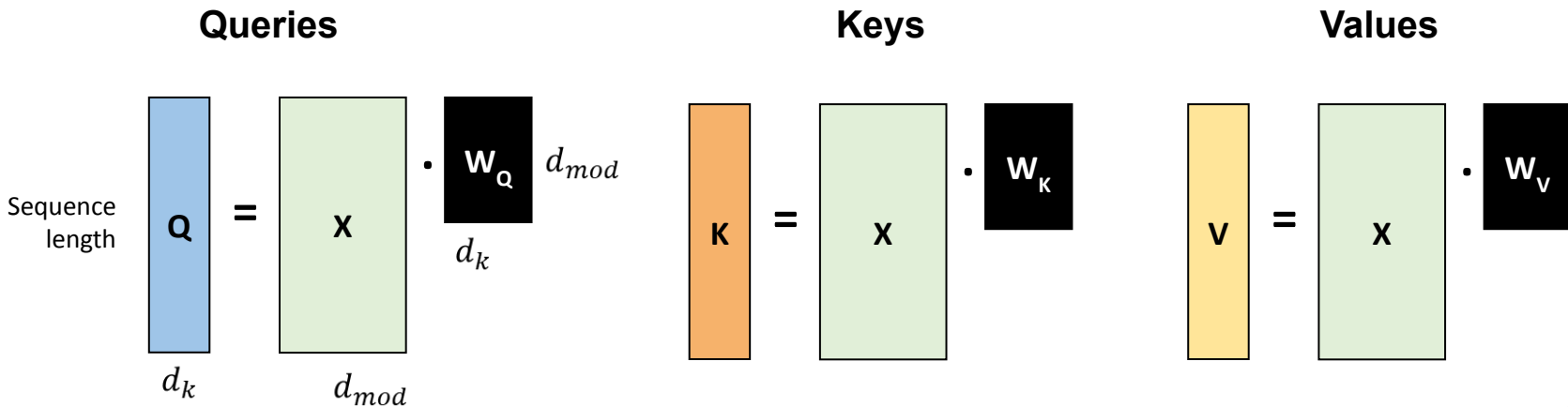
The output embedding is a weighted combination of all the input embeddings to the representation of the current position.



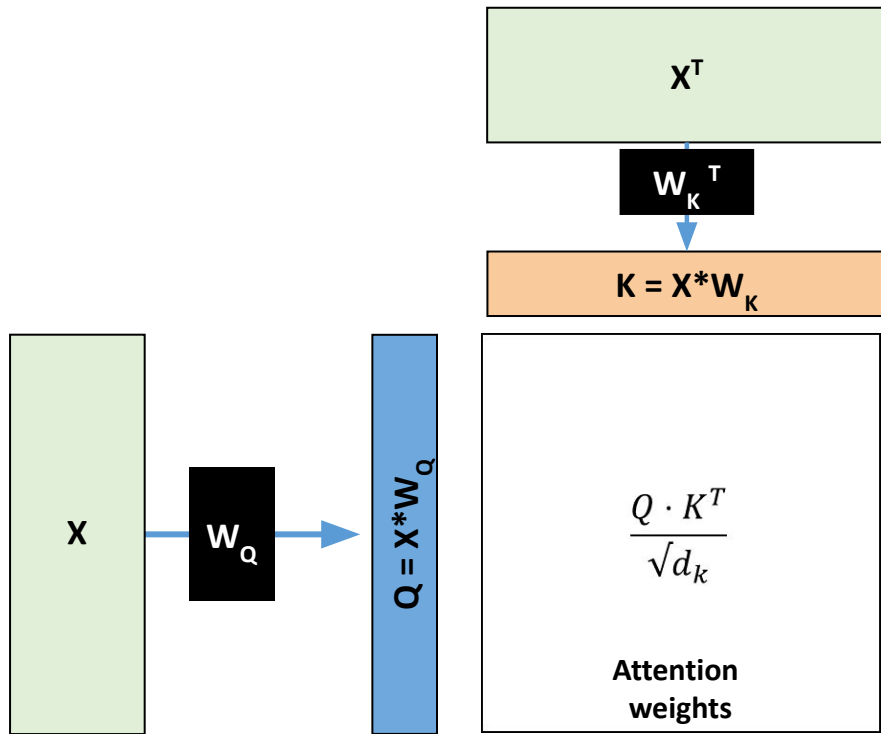


Self-Attention: Let's break it down

Given a sequence of token embeddings $\mathbf{X}$, the model computes **queries (Q)**, **keys (K)**, and **values (V)** by applying learned linear transformations:



Self-Attention: Let's break it down



For each token in X :

We **query** the keys of all tokens (**keys**) and compute their **attention weights**.

The token will be the value of all tokens **weighted** by their corresponding attention.



Scaled Dot-Product Attention

$$X \cdot W_Q = Q$$

Diagram illustrating the calculation of the Query matrix Q . A green vertical rectangle X is multiplied by a black square matrix W_Q to produce a blue vertical rectangle Q . The word "quick" is highlighted in blue at the top of Q .

$$X^T \cdot W_K = K$$

Diagram illustrating the calculation of the Key matrix K . A green horizontal rectangle X^T is multiplied by a black square matrix W_K to produce an orange horizontal rectangle K . The words "brown" and "dog" are highlighted in orange at the ends of K .

The words "The quick brown fox jumps over the lazy dog" are listed below K .

Diagram illustrating the calculation of the Normalized attention weights. A large box contains the formula $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$ and the text "Normalized attention weights". A blue bar highlights the first row of this matrix, corresponding to the word "quick".

$$X^T \cdot W_V = V$$

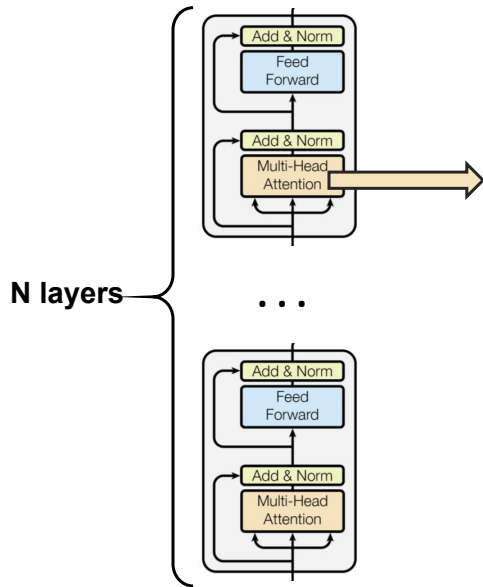
Diagram illustrating the calculation of the Value matrix V . A green horizontal rectangle X^T is multiplied by a black square matrix W_V to produce a yellow vertical rectangle V .

Diagram illustrating the final output. A green vertical rectangle head_i is shown, with the word "quick" highlighted in blue at the top. Arrows indicate that the first row of the Normalized attention weights matrix is multiplied by the Value matrix V to produce the output head_i .

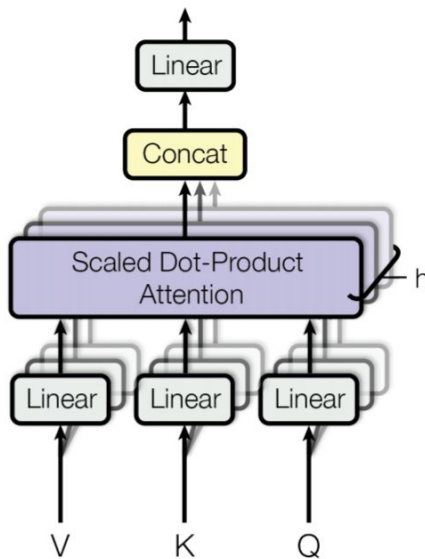


Encoder Layers: Multi-Head Attention

Transformer Layer



Multi-Head Attention



Multi-head attention is an **extension** of the self-attention mechanism used in Transformer models.

- It allows the model to focus on different parts of the input sequence simultaneously.
- Provides a richer and more nuanced modeling of the data.

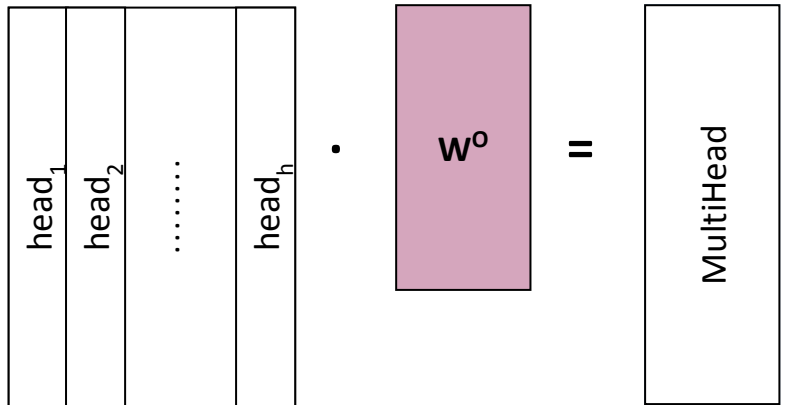


Encoder Layers: Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

W^O

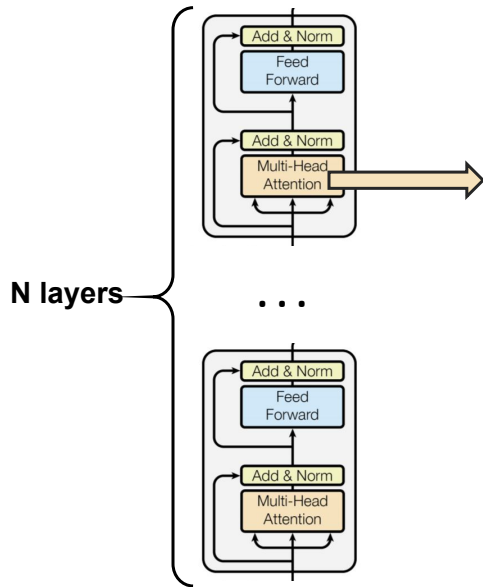
weight matrix that projects the **concatenated vector** back into the original embedding space



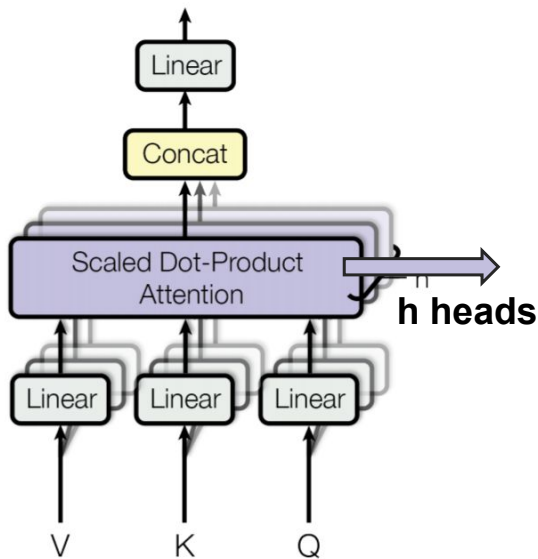


Encoder Layers: Overview

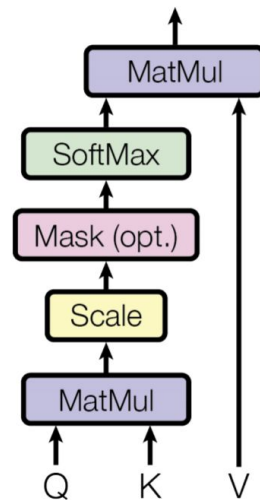
Transformer Layer



Multi-Head Attention

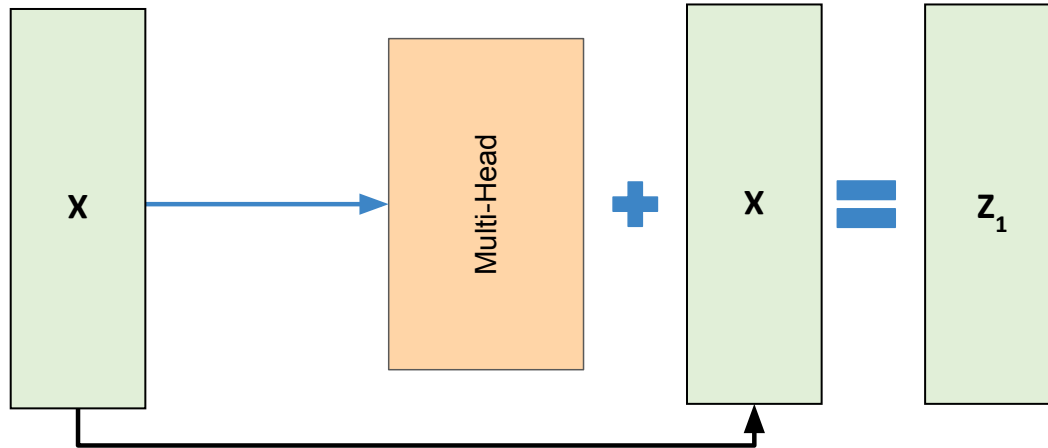


Single-Head Attention





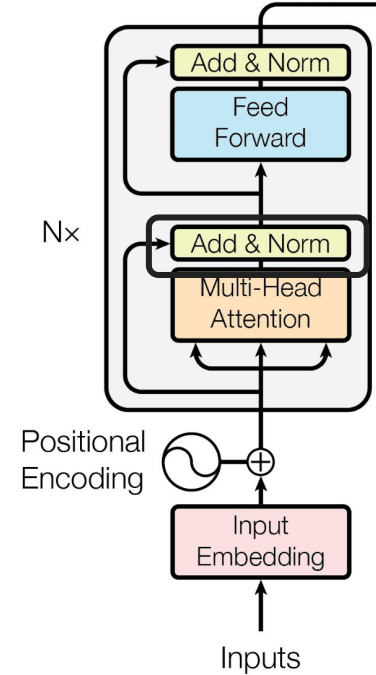
Encoder Layers: Add & Norm



Residuals: Helps retain the original input information (input and positional)

Normalization:

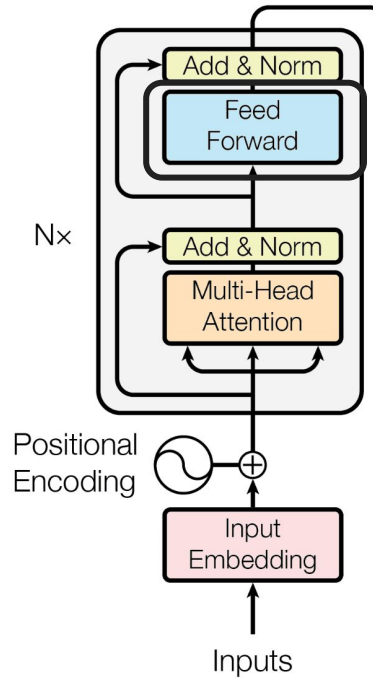
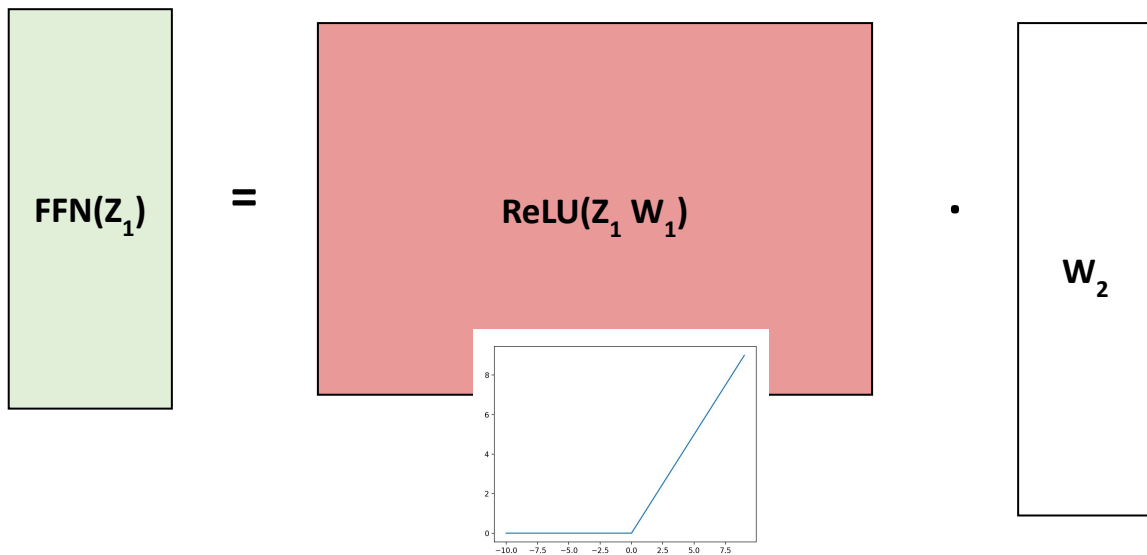
Stabilizes the learning process by ensuring the outputs have a consistent scale and distribution.





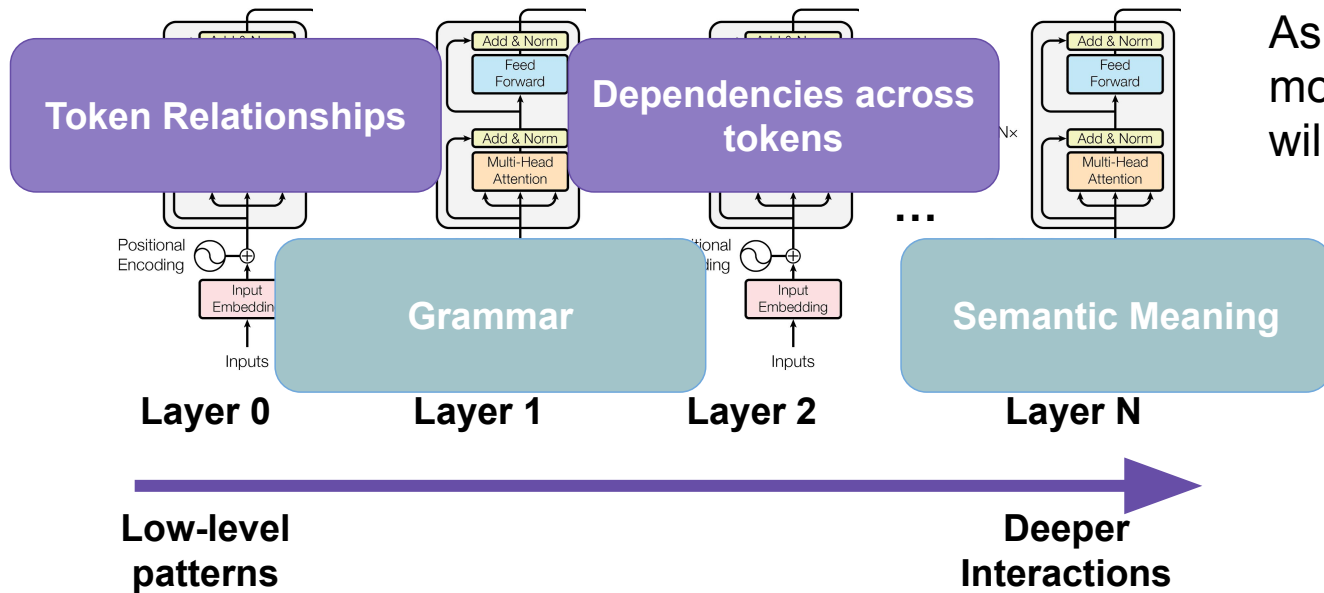
Encoder Layers: Feed Forward

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$





Multiple Layers... Multiple interactions!

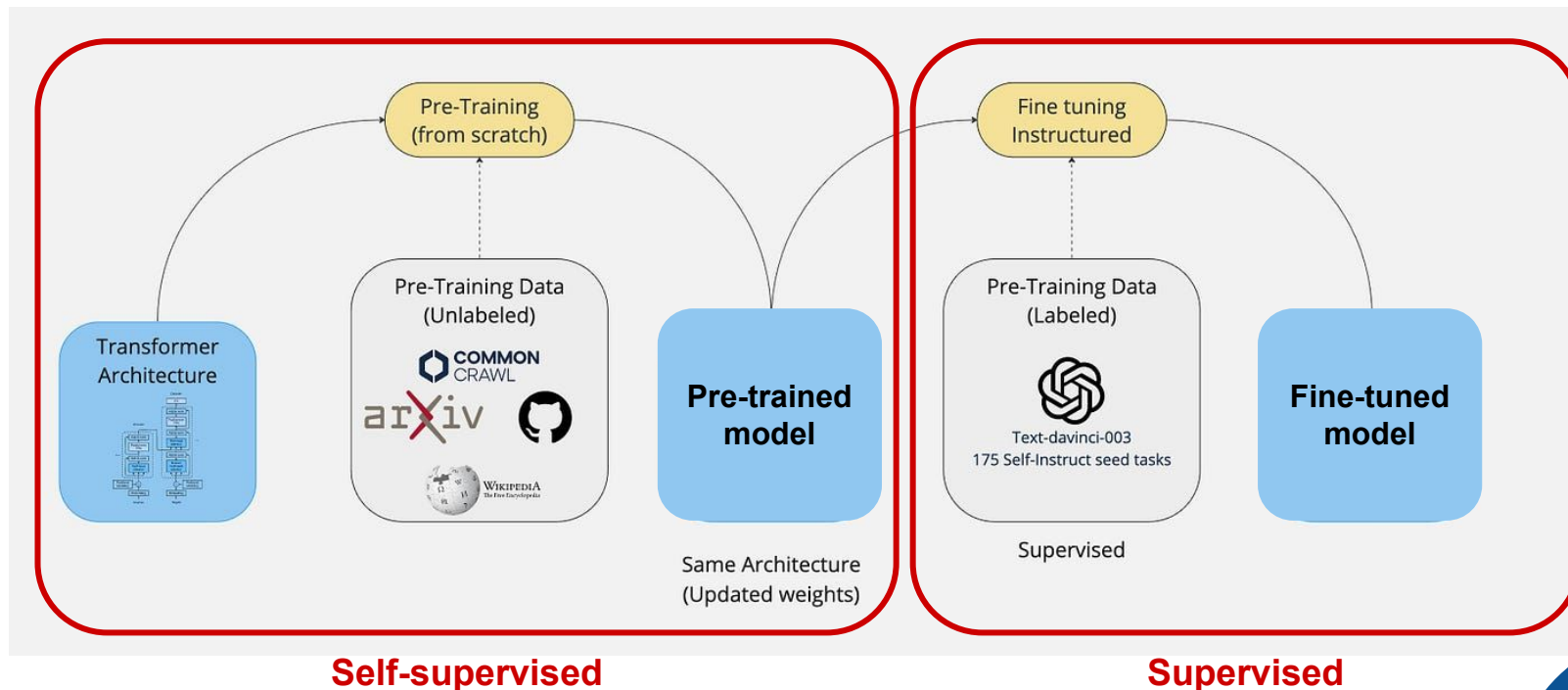


As we go up in layers, more abstract interactions will be captured!

Chapter 9.

The infamous “*Pre-train once,
fine-tune many times*” paradigm
was born!

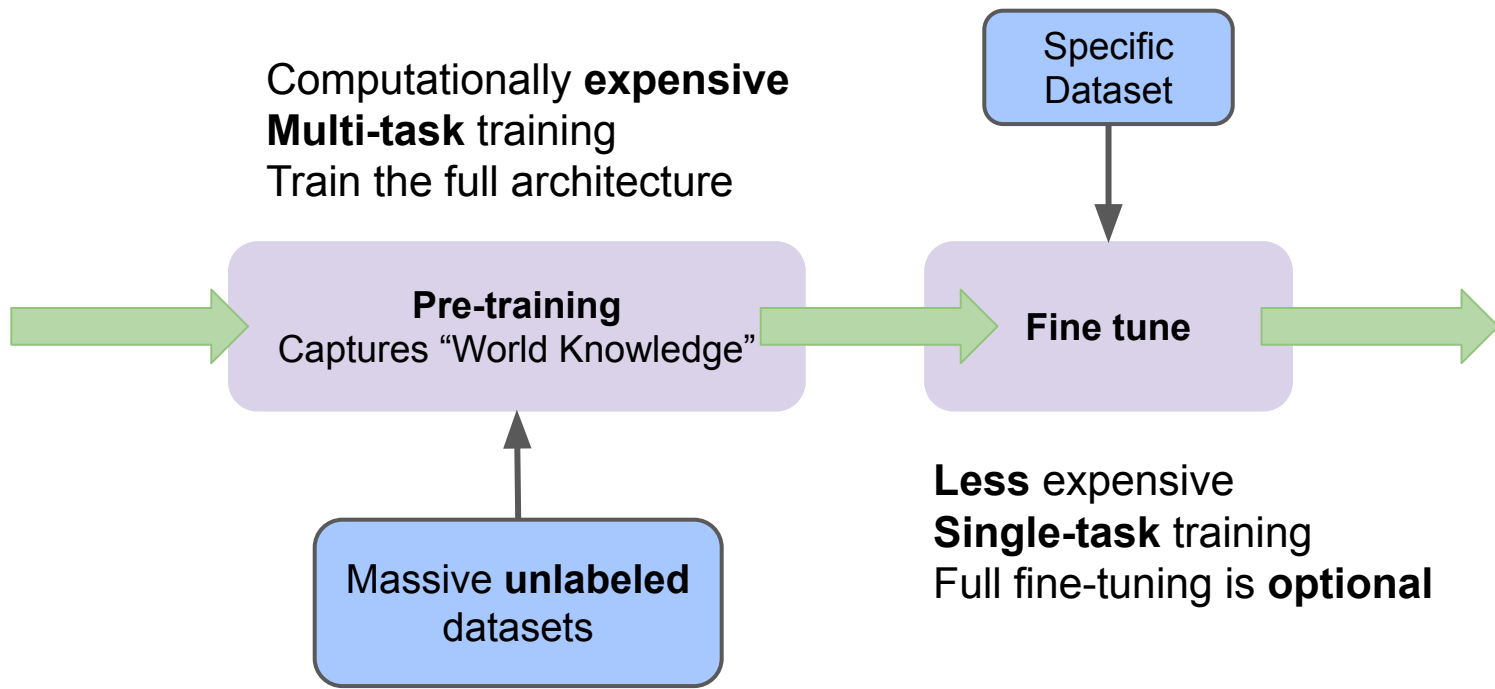
The new normal!



Self-supervised

Supervised

A very (VERY) common pipeline



Pre-training: Capturing World Knowledge

The secret: **Self supervised learning**

The model creates **labels** from the **data** itself!

- **No need** for large amounts of annotated data, which requires:
 - Time
 - Human resources
 - Money

The data itself provides the supervision signal!

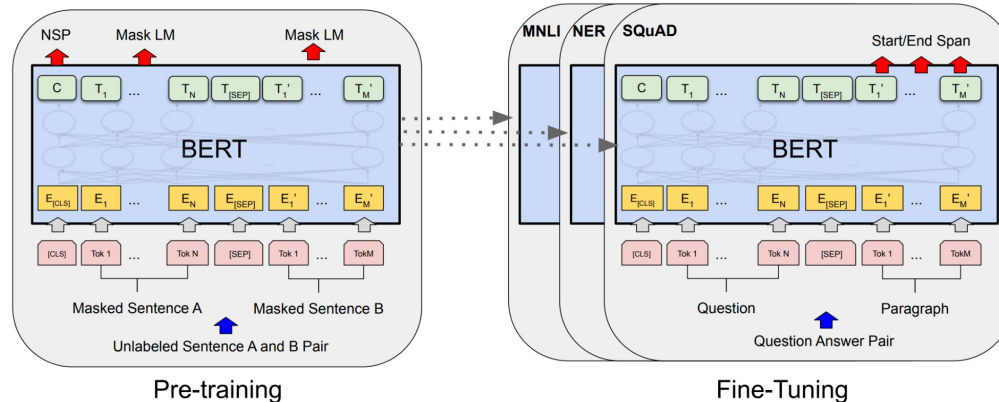


BERT: Bidirectional Encoder Representations from Transformers

What is BERT?

A transformer encoder-model that popularized the **Pre-train;Fine-tune** paradigm.

- Learned representations based on **bidirectional context**

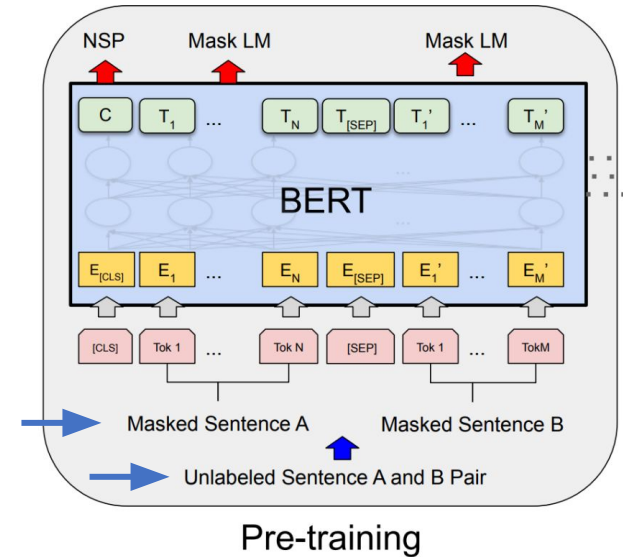




BERT: Bidirectional Encoder Representations from Transformers

Pre-training Tasks:

1. Masked Language Modeling
2. Next Sentence Prediction



Masked Language Modeling

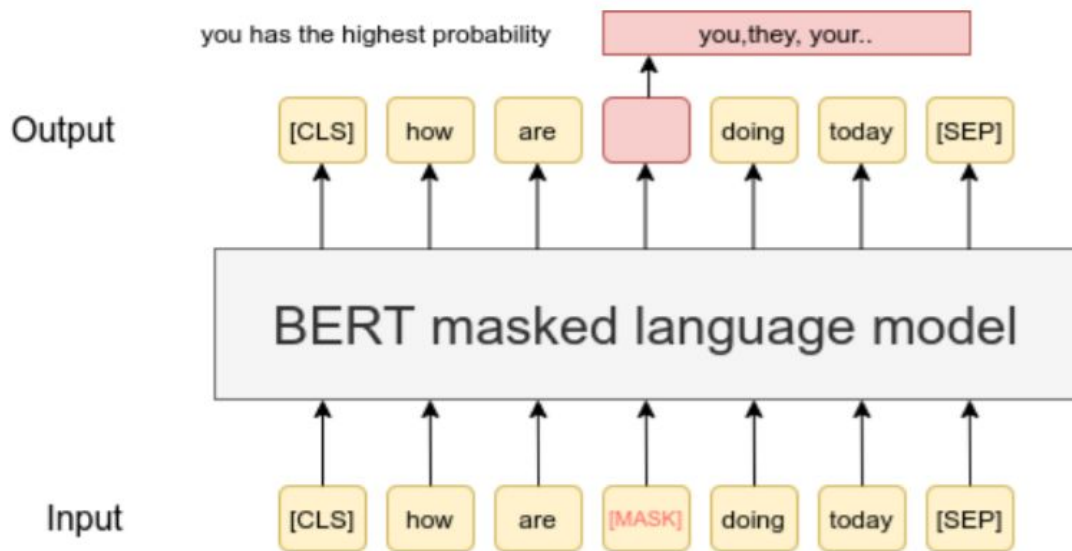
Learning Contextual Relationships:

- In natural language, a word's meaning is heavily influenced by the surrounding words.
- **Masked Language Modeling (MLM)** helps the model learn these relationships by predicting masked words based on their context.

Bidirectional Understanding

MLM allows the model to understand a **word's meaning** by considering both its **left and right context**

Masked Language Modeling



[CLS]: Classification Token

- Added at the beginning of every input
- Summary of the entire sequence

[SEP]: Separator Token

[MASK]: Masking Token

Use the output of the masked word's position to predict the masked word

Next Sentence Prediction

Learning if two sentences are logically connected:

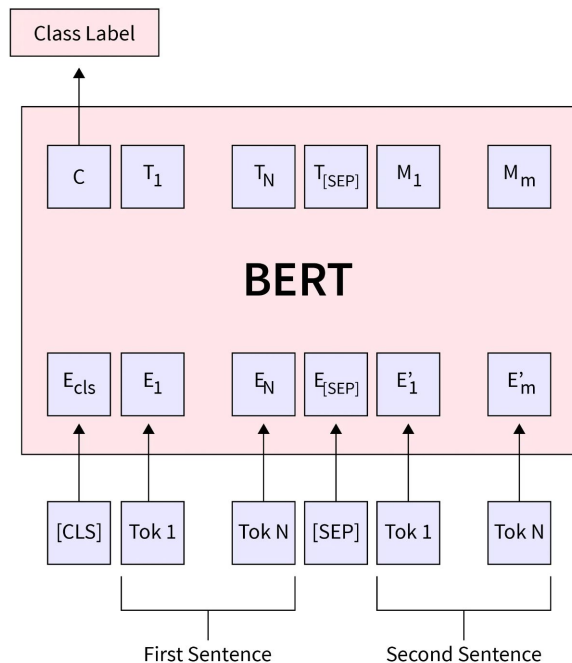
- In natural language, **topic continuity** and **discourse-level** analysis is incredibly important.
- **Next Sentence Prediction (NSP)** helps the model develop contextual understanding by capturing the relationship between consecutive sentences

Question-Answering

Document
Classification

Information Retrieval

Next Sentence Prediction



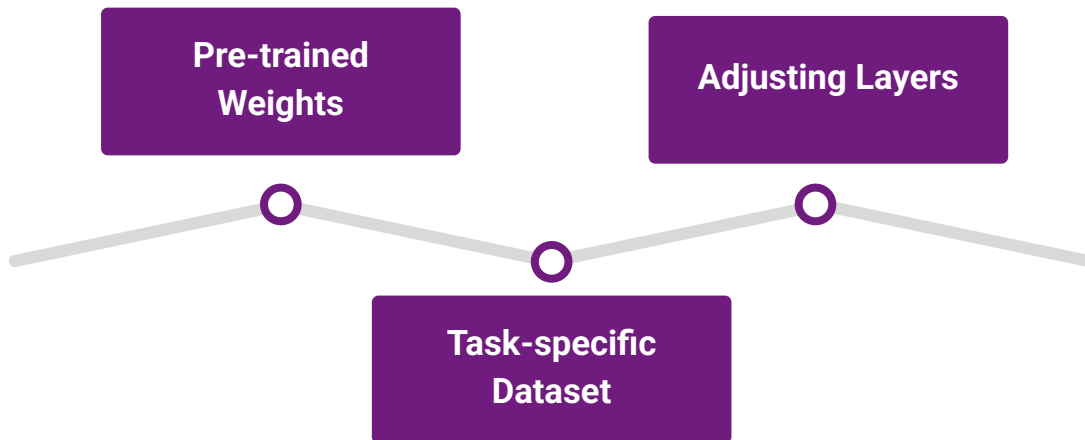
Is sentence B the actual next sentence after sentence A?

The CLS token learns to represent the meaning of the entire sequence and can be used to classify it!

It's a **binary classification** task using the [CLS] token

Fine-tuning: The ultimate recipe

This stage allows to specialize the model's knowledge for a **particular task** **without starting from scratch**



Fine-tuning: Tasks

1. Document Classification: Assign a label to a sequence of text

- Sentiment Analysis
- Topic classification
- Hate-speech detection

Use the **[CLS]** token
embedding

Add a **linear/softmax layer**

1. Named Entity Recognition: Assign a label to each token

- Person, Organization, Location extraction
- Legal and medical entities extraction

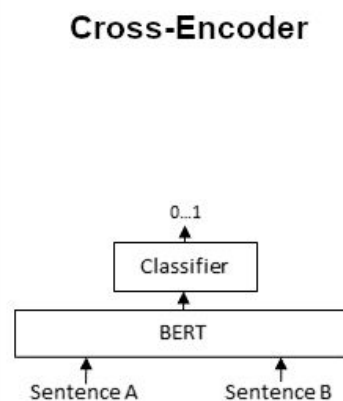
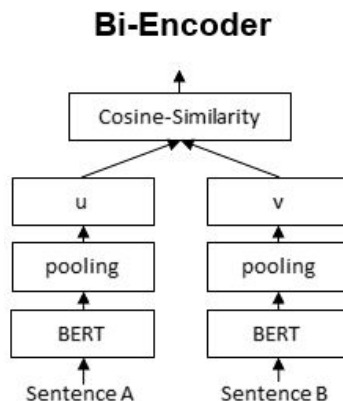
Token level classifier on top of the encoder outputs

Fine-tuning: Tasks

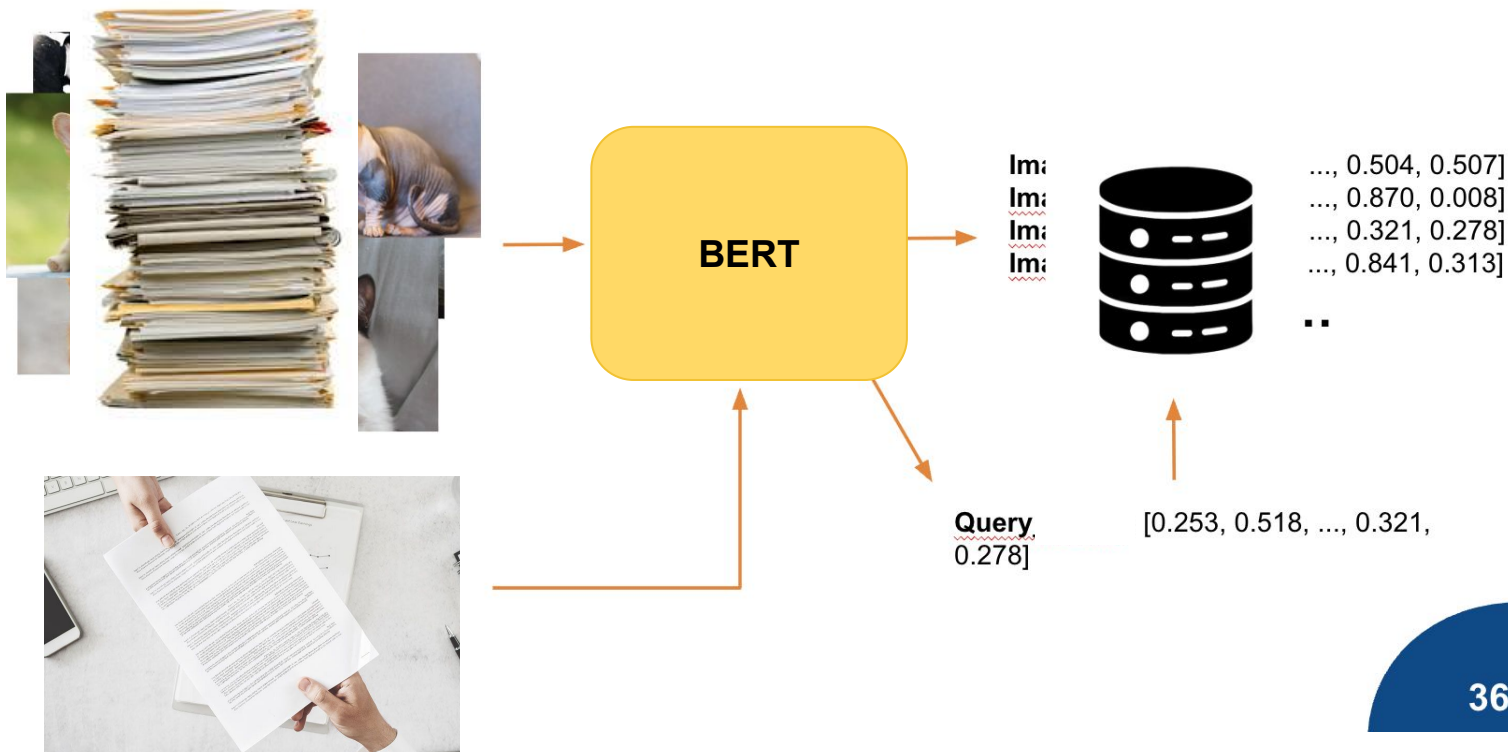
1. **Question Answering (Span Extraction):** Extract a span of text that answer the question

Predict **start** and **end** token positions

1. **Query Document Retrieval:** Extract a span of text that answer the question



Search



Exercise Time

